



うさうさ研修工房 | AIツール開発プロンプト集

ボタンを押したら資料がパツと出てくる「単一HTML生成ツール」を、AIと一緒に作る・直す・テストするための再利用プロンプト集です。コピーして `<>` の中を埋めるだけで使えます。

設計思想: WHY→WHAT→HOW / 人ではなく仕組み / 事実と所感を分ける / 小さく頼んで育てる。

目次

1. [使い方](#)
 2. [共通ルール\(最初に貼る\)](#)
 3. [新規ツールを作る](#)
 4. [機能を1つ追加する](#)
 5. [出力フォーマットを足す\(md/CSV/PDF\)](#)
 6. [テストを書いてもらう](#)
 7. [エラーを直す](#)
 8. [リファクタ・整理](#)
 9. [文章・UIコピーを整える](#)
 10. [ライセンス](#)
-

使い方

- 各プロンプトの `` ``` の中をコピー → `<...>` を自分の内容に置換 → AIに貼る
 - まず「0. 共通ルール」を会話の最初に貼ると、以降の出力が安定します
 - 一度に大盛りを頼まず、1.→2.→4. の順で小さく育てるのがコツ
-

0. 共通ルール(最初に貼る)

あなたは一流のフロントエンドエンジニアで、教育設計にも詳しいパートナーです。

これから「単一HTMLの生成ツール」を一緒に作ります。次のルールを常に守ってください。

【技術ルール】

- 出力は単一のHTMLファイル1つ（HTML/CSS/JSを内包）。外部CDN・外部フォント・外部APIに依存しない（オフラインで動く）

- localStorage等のブラウザストレージは使わない。状態はJSの変数で持つ

- バニラJSのみ（フレームワーク不要）。古い書き方や未確認のAPIは使わない

- 画面の右下に出る自動セルフテスト（起動時実行）を内蔵し、結果を表示する

- 生成したコードは自分で見直し、明らかな不具合は直してから出す

【書き方ルール】

- 目的（WHY）→ 作るもの（WHAT）→ 手順（HOW）の順で考える

- 変更点は簡潔に説明。コード全体を毎回貼り直すのではなく、差分が分かる形で

【守りたい価値観】

- 「人ではなく仕組み」：ミスは個人を責めず、仕組みで防ぐ

- 事実と所感を分けて説明する

- 固有名（社名・人名）は出さず《》や役割名で扱う

準備ができたなら「OK」とだけ返してください。

1. 新規ツールを作る

次の生成ツールを、共通ルールに沿って単一HTMLで作ってください。

- ツール名：<例>研修カリキュラム ジェネレーター</>

- 目的（WHY）：<例>未経験講師でも、品質の揃ったカリキュラムをすぐ作れる</>

- 使う人（WHO）：<例>文系・未経験の新人講師</>

- 入力（INPUT）：<例>コース名（必須）・対象・ゴール・コマ数</>

- 出力（OUTPUT）：<例>見出し＋表＋チェックリストのMarkdown。画面表示も</>

- テーマ/見た目:<例>落ち着いた配色・絵文字は控えめ・動きは任意>

- 制約:<例>単一HTML・オフライン・外部依存なし>

まずは「骨組み(入力欄+生成ボタン+出力エリア+セルフテスト枠)」だけ作ってください。

中身のテンプレ文面は次の指示で足します。完成したら、何を確認すればいいかも教えてください。

2. 機能を1つ追加する

(既存のツールに対して)次の機能を1つだけ追加してください。他の挙動は変えないこと。

- 追加する機能:<例>「対象キャリア」を選ぶと出力に専用セクションが付く>

- 入力UI:<例>チップ(新卒/第二新卒/現役/フリーランス・複数選択可)>

- 出力への反映:<例>「対象キャリア別の調整」セクションを追記>

- エッジケース:<例>未選択なら何も出さない>

追加後、この機能を確認める内蔵セルフテストも1~2個足してください。

変更点を簡潔に説明してから、必要な箇所だけ差分で示してください。

3. 出力フォーマットを足す(md/CSV/PDF)

生成結果を次の形式でダウンロードできるボタンを追加してください。共通ルール準拠。

- Markdown(.md): そのまま

- CSV: 表は見出しラベル付き、文字化け防止にBOMを付ける

- メモ(.txt): 見出しを ■ ● などに整形した読みやすいテキスト

- HTML(.html): 単体で開ける簡易スタイル付き

- PDF: 印刷ダイアログ(window.print)を使い、ファイル名をタイトルに反映

- 一式: 上記をまとめて連続ダウンロード

ファイル名は「<基準名>_YYYYMMDD_HHMM」。任意のファイル名入力があればそれを優先（記号はサニタイズ）。

各ボタンに対応する内蔵セルフテストも足してください。

4. テストを書いてもらう

このツールの「商用テスト」をNode.js(jsdom)で書いてください。目的は、出荷前に壊れていないかを自動で確かめることです。

- jsdomでHTMLを読み込み、起動時の内蔵セルフテストの完了を待ってから検証する

- 検証する観点：

- 主要な生成パターンがすべて壊れず出力される(undefined/NaNが出ない)

- 必須入力のバリデーション

- 各ダウンロード(md/CSV/メモ/html/一式)が想定どおり呼ばれる

- CSVがBOM付き・スキーマ準拠

- 入力値のXSSエスケープ(などが実体化しない)

- ページ実行中にエラーが出ていない

- 結果はログファイルに「PASS / FAIL」で出し、全合格なら exit 0

- ブラウザにしか無いAPI(print, Blob, scrollIntoView 等)は必要に応じてスタブする

書けたら、5回連続で実行して安定して全合格することを確認する手順も教えてください。

5. エラーを直す

次のエラー(と状況)を直してください。原因→直し方→修正差分の順で、簡潔に。

- 何をしたら出たか:<例>「生成」ボタンを押した直後>

- どこまで動くか:<例>画面は表示されるがボタンが無反応>

- エラーメッセージ(そのまま貼る):

<ここにコンソールの赤い文字を丸ごと貼る>

推測で大きく書き換えず、原因を切り分けて最小限の修正にしてください。

直したら、同じ問題を防ぐ内蔵セルフテストを1つ足してください。

6. リファクタ・整理

挙動を一切変えずに、次の観点で整理してください(リグレッション厳禁)。

- 重複している処理を共通関数にまとめる

- 命名を分かりやすくする

- 役割ごとにコードの並びを整える(装飾 / 生成ロジック / 変換 / 出力 / イベント / テスト)

変更後、内蔵セルフテストと商用テストが**すべて従来どおり通る**ことを前提にしてください。

変えた点を箇条書きで説明してください。

7. 文章・UIコピーを整える

このツールの画面の文言(ラベル・説明・トースト)を、次の読者に合わせて整えてください。

- 読者:<例>文系・未経験の社会人>

- トーン:<例>やさしく・前向き・専門用語は最初にかみ砕く>

- ルール:固有名は出さない／事実と所感を分ける／プレッシャーを与えない言い回し

挙動は変えず、文言だけ差し替えてください。変更箇所を一覧で示してください。

ライセンス

社内・個人利用を想定したテンプレート集です。利用時は各社のセキュリティ方針（機密情報をAIに入力しない等）に従ってください。

 面白きこともなき世を面白く